

Prompt ที่ใช้เทรน t01

คำสั่งที่ส่งไปกับทุกภาพ — มันเขียนว่าอะไร ทำไมถึงเขียนแบบนั้น และต้องใช้ยังไงตอนเรียกโมเดล

Training repo · `tune_ai/t01/` · โมเดล `Qwen3.6-35B-A3B` · สร้างเอกสาร 2026-08-03

1. Prompt คืออะไรในชุดข้อมูลนี้

ตอนเทรนโมเดลอ่านแบบก่อสร้าง เราไม่ได้ป้อนแค่ "ภาพ → คำตอบ" เฉย ๆ แต่ป้อนเป็น บทสนทนา 1 รอบ คือ

```
user      : [ภาพหน้าแบบ 1 รูป] + คำสั่ง (prompt)
assistant : {"png":"31","views":[ ... ]}      ← JSON คำตอบที่คนตรวจแล้ว
```

ส่วนที่เรียกว่า **prompt** คือข้อความที่แนบไปหลังภาพ — เป็นตัวบอกโมเดลว่า "เห็นภาพนี้แล้วให้ทำอะไร ตอบออกมาหน้าตาแบบไหน และมีกฎอะไรห้ามพลาดบ้าง" โมเดลจะเรียนว่า *เมื่อเจอคำสั่งชุดนี้ + ภาพแบบนี้ ต้องตอบ JSON แบบนี้*

ประเด็นสำคัญที่สุดของทั้งเอกสารนี้: prompt ตอนเทรนกับตอนใช้งานจริง ต้องเป็นข้อความเดียวกันเป๊ะทุกตัวอักษร ถ้าเปลี่ยนแม้แต่นิดเดียว โมเดลจะเจอ "คำสั่งที่ไม่เคยเห็น" แล้วผลจะเพี้ยนทันที

ทั้งชุดมี prompt แค่ตัวเดียว

ทั้ง 403 ตัวอย่าง (train 315 + val 88) ใช้ prompt **ข้อความเดียวกันหมด** ไม่มี system prompt แยก ไม่มีคำสั่งเฉพาะหน้า ไม่มีการบอกว่า "หน้านี่เป็น plan นะ" หรือ "หน้านี่เป็น section นะ"

เหตุผล: ตอนใช้งานจริง เราไม่รู้ล่วงหน้าว่าหน้าที่ยื่นให้โมเดลเป็นหน้าอะไร ถ้าเทรนโดยบอกไว้ล่วงหน้า พอถึงเวลาใช้จริงจะบอกไปไม่ได้ = โมเดลเจอสถานการณ์ที่ไม่เคยฝึก

1	~1,022	403	5
prompt ตัวเดียวทั้งชุด	token ต่อ 1 ตัวอย่าง	ตัวอย่างที่ใช้ prompt นี้	ตัวที่มีส่วนต่อท้ายพิเศษ

เก็บอยู่ในตัวแปร `PROMPT_SHORT` ที่ `tune_ai/t01/data_before_tune/build_dataset.js` บรรทัดที่ 80 และมีสำเนาข้อความล้วนที่ `tune_ai/t01/ai_output_บ้าน_ใหญ่_1ชั้น_01/_prompt_short.txt`

2. เนื้อ prompt ตัวจริง (ต้นฉบับภาษาอังกฤษ)

นี่คือข้อความเต็มที่ส่งไปกับทุกภาพ — เขียนเป็นภาษาอังกฤษล้วน แม้ว่าตัวแบบก่อสร้างและคำตอบจะเป็นภาษาไทย

You are reading one page of a Thai reinforced-concrete (RC) construction drawing set.

Extract everything on the page into JSON following the `primary_rawjson_schema`.

Inventory EVERY view/box on the page first, then emit one entry per view in "views" (a single-view page still uses a one-entry array – never drop a view). Each view carries its own "pattern": plan, section, schedule, notes, index, material_list, site_plan, side_profile, gridline, title, symbol, roof_plan, misc, or unknown.

GRID AND DUMMY GRID – the single most error-prone part of this task, read carefully:

- `grid_ref` reads row-letter first, then column ("A-1", not "1-A"). Point-type elements (footing/column) use a `grid_refs` array instead of start/end.
- A structural line not on a named/printed grid still needs a name: append a prime to the nearest named grid ("1'", "A'"). If more than one dummy line falls in the same gap, number them in reading order (left→right / top→bottom): 1st gets one prime, 2nd gets two.
- THE KEY RULE: if a beam's start or end point does not sit on any grid line you can see, that point still needs a grid line – it does NOT mean the beam should be dropped. Trace every beam segment, including short stubs near stairs/closets. For each endpoint: use the existing named/dummy grid if one is there; if not, read its position off a printed dimension chain and record a new dummy grid, then reference the beam against it. Never: (a) drop the beam because it "isn't on the grid", (b) write a prose description instead of `grid_ref_start/grid_ref_end`, (c) set start=end with a null span. Exception: a slab/eave edge with no beam label and no corner columns is not structural and needs no dummy grid.
- Span length comes from the grid table, not your own visual estimate.

REBAR (`main_bar`):

- Always split top/bottom, even when the counts are equal – never collapse into one.
- If a section shows a clearly distinct row of bars at mid-depth (own leader line, sitting between the top and bottom rows, usually a deep beam), record it as a third face, `main_bar.middle`. Do not fold it into `additional_bars`, and do not invent one by splitting a top/bottom cluster.
- A circle symbol ($\emptyset$) always means round bar (RB); visible ribs mean deformed bar (DB)
 - read the symbol, never infer type from diameter.
- Columns use a single `main_bar.count` for the 4 corner bars – do not split top/bottom.
- Before assigning an "additional" bar to top or bottom, check the leader line itself, not just the label wording – the same-looking label has resolved to opposite sides on different marks in this series.

OUTPUT DISCIPLINE:

- Same `element_id` appearing more than once on this page with non-overlapping

positions → merge into one entry (sum count, concatenate grid_refs). Exception: a multi-level schedule keeps the same element_id per level as separate entries, using

a "level" field – never embed the level into element_id.

- One atomic entry per grid-to-grid beam segment; do not pre-group same-mark spans.
- Reading order: top-to-bottom by row, left-to-right by column, vertical before horizontal at a shared start point.
- Use null for anything unclear. Do not guess or invent a value.

Reply with JSON only. No markdown fence, no commentary.

3. อธิบายทีละบล็อก (ฉบับภาษาไทย)

prompt แบ่งเป็น 6 บล็อก เรียงจากกว้างไปแคบ: บอกงาน → บอกโครงสร้างคำตอบ → บอกกฎที่พลาดบ่อยที่สุดก่อน → กฎรอง → วินัยการตอบ → ปิดท้าย

บล็อก 1 บทบาทและงาน — 2 บรรทัด

"You are reading one page of a Thai RC construction drawing set. Extract everything into JSON following the primary_rawjson_schema."

แปล: "คุณกำลังอ่านแบบคอนกรีตเสริมเหล็กของไทย 1 หน้า ให้สกัดทุกอย่างบนหน้านั้นออกมาเป็น JSON ตามสเปค primary_rawjson_schema"

- ระบุชัดว่า "1 หน้า" — ไม่ใช่ทั้งเล่ม ไม่ใช่ 1 view โมเดลต้องมองทั้งหน้า
- ระบุว่าเป็นแบบ ไทย — เพื่อให้โมเดลเตรียมอ่านตัวอักษรไทยและหน่วยแบบไทย
- อ้างอิงสเปค primary_rawjson_schema แต่ไม่ได้แนบตัวสเปคมาด้วย — เหตุผลอยู่ในหัวข้อ 5

บล็อก 2 นับ view ให้ครบก่อน — 4 บรรทัด

แปล: "ให้กวาดนับทุก view / ทุกกล่องบนหน้านีก่อน แล้วค่อยออกผลเป็น 1 entry ต่อ 1 view ในอาร์เรย์ views หน้าที่มี view เดียวก็ยังคงเป็นอาร์เรย์ที่มีสมาชิก 1 ตัว — ห้ามทิ้ง view เด็ดขาด แต่ละ view ต้องมี pattern ของตัวเอง"

แล้วตามด้วยรายชื่อ pattern ที่เลือกได้ 14 ค่า:

plan ผังพื้น	section รูปตัด	schedule ตารางรายการ	notes ข้อกำหนด
index สารบัญแบบ	material_list รายการวัสดุ	site_plan ผังบริเวณ	side_profile รูปด้าน
gridline เส้นกริด	title หน้าปก	symbol สัญลักษณ์	roof_plan ผังหลังคา
misc เบ็ดเตล็ด	unknown ไม่รู้	มี unknown ไว้ให้โมเดลยอมรับว่าไม่รู้ ดีกว่าบังคับให้เดา	

ทำไมต้องบังคับให้เป็นอาร์เรย์เสมอ — แม้หน้าที่มี view เดียว เพราะถ้าบางหน้าตอบเป็นอ็อบเจกต์เดี่ยว บางหน้าตอบเป็นอาร์เรย์ โค้ดปลายทางจะต้องเช็ค 2 กรณีตลอด รูปทรงคำตอบที่คงที่ = โค้ดที่อ่านผลง่ายกว่ามาก

บล็อก 3 GRID และ DUMMY GRID — ยาวที่สุด ~40% ของ prompt

บล็อกนี้ประกาศตรง ๆ ในตัว prompt เองว่า "the single most error-prone part of this task, read carefully" = "เป็นส่วนที่พลาดบ่อยที่สุดของงานนี้ อ่านให้ดี ๆ"

กฎ	ความหมาย
ลำดับการอ่าน grid_ref	อ่านตัวอักษรแถวก่อน แล้วค่อยตัวเลขคอลัมน์ → "A-1" ไม่ใช่ "1-A" ขึ้นส่วนแบบจุด (ฐานราก / เสา) ใช้อาร์เรย์ <code>grid_refs</code> แทน <code>start / end</code>
Dummy grid (เส้นกริดที่แบบไม่ได้ตั้งชื่อไว้)	เส้นโครงสร้างที่ไม่อยู่บนกริดที่มีชื่อพิมพ์ไว้ ก็ยังต้องมีชื่อ — ให้เติมเครื่องหมาย prime ต่อจากกริดที่ใกล้ที่สุด เช่น <code>1'</code> , <code>A'</code> ถ้าในช่องเดียวกันมีหลายเส้น ให้นับตามลำดับการอ่าน (ซ้าย→ขวา / บน→ล่าง) เส้นแรก 1 ซิต เส้นที่สอง 2 ซิต (<code>1''</code>)
THE KEY RULE (กฎที่เขียนตัวใหญ่)	ถ้าปลายคานไม่ตกบนเส้นกริดที่มองเห็น — จุดนั้นยังต้องมีกริดอยู่ดี ไม่ได้แปลว่าให้ทั้งคานทิ้งไป ให้ไล่ทุกช่วงคาน รวมถึงคานสั้น ๆ แถวบันไดหรือตู้เก็บของ วิธีทำ: ถ้ามีกริด (มีชื่อหรือ dummy) อยู่แล้วก็ใช้ตัวนั้น ถ้าไม่มีให้อ่านตำแหน่งจากโซ่เส้นบอกระยะที่พิมพ์อยู่บนแบบ แล้วบันทึกเป็น dummy grid ตัวใหม่ จากนั้นค่อยอ้างอิงคานกับกริดตัวนั้น
ความยาวช่วง (span)	เอามาจากตารางกริด ไม่ใช่กะเอาจากสายตา

เทคนิคเขียน prompt ที่ใช้ตรงนี้ — เขียนเป็น "ห้ามทำ" ไม่ใช่แค่ "ให้ทำ"

หลังกฎ THE KEY RULE มีรายการห้ามชัด ๆ 3 ข้อ:

- (a) ห้ามทั้งคานเพราะ "มันไม่อยู่บนกริด"
- (b) ห้ามเขียนบรรยายเป็นประโยค แทนที่จะใส่ `grid_ref_start / grid_ref_end`
- (c) ห้ามตั้ง `start` เท่ากับ `end` แล้วปล่อย span เป็น null

ทั้ง 3 ข้อนี้คือ พฤติกรรมที่ไม่เคยเคยทำจริง ตอนสเก็ตรอบก่อน ๆ — ไม่ได้คิดขึ้นมาลอย ๆ การบอก "ห้ามทำแบบนี้" ได้ผลกว่าบอกแค่ "ให้ทำแบบนี้" เพราะมันปิดทางออกที่ไม่เคยชอบเลือกเวลาจูนมูม

และมีข้อยกเว้นกันโมเดลทำเกิน — ขอบสแลบหรือขายคาที่ไม่มีมาร์คคานกำกับ และไม่มีเสาที่มุม ถือว่าไม่ใช่โครงสร้าง
ไม่ต้องสร้าง dummy grid ให้

ถ้าไม่เขียนข้อยกเว้นนี้ โมเดลจะเอากฎ "ทุกอย่างต้องมีกริด" ไปใช้กับเส้นขอบทุกเส้นบนแบบ แล้วสร้าง dummy grid มั่ว
เต็มไปหมด

บล็อก 4 REBAR — เหล็กเสริม ~20% ของ prompt

กฎ	ความหมาย และเหตุผล
แยกบน/ล่างเสมอ	ต้องแยก <code>main_bar.top</code> กับ <code>main_bar.bottom</code> ตลอด แม้จำนวนเส้นจะเท่ากัน ห้ามยุบรวมเป็นตัวย่อย เพราะถ้ายุบตอนบน 4 ล่าง 4 เป็น "8 เส้น" ข้อมูลว่าเหล็กอยู่หน้าไหนหายไปเลย ซึ่งเป็นข้อมูลที่ใช้ทำ BBS จริง
เหล็กแถวกลาง <code>main_bar.middle</code>	ถ้ารูปตัดแสดงแถวเหล็กที่กลางความลึกชัดเจน (มีเส้นชี้ของตัวเอง อยู่ระหว่างแถวบนกับล่าง — มักเจอในคานลึก) ให้บันทึกเป็นหน้าที่สาม ห้ามยัดรวมเข้าไปใน <code>additional_bars</code> และ ห้ามสร้างขึ้นมาเองด้วยการแยกกลุ่มเหล็กบน/ล่างออกเป็นสอง
อ่านสัญลักษณ์ ไม่ใช่เดาจากขนาด	สัญลักษณ์วงกลม (Ø) = เหล็กกลม RB เสมอ · เห็นครีป = เหล็กข้ออ้อย DB ให้อ่านจากสัญลักษณ์ ห้ามอนุมานชนิดจากเส้นผ่านศูนย์กลาง เป็นความผิดพลาดคลาสสิกของ VLM — เห็น Ø12 แล้วเดาว่าเป็น DB12 เพราะขนาด 12 มม. "ปกติจะเป็นข้ออ้อย"
เสาใช้ count เดียว	เสาใช้ <code>main_bar.count</code> ตัวเดียวสำหรับเหล็ก 4 มุม ห้ามแยกบน/ล่าง เพราะเสาตั้งในแนวดิ่ง ไม่มีแนวคิด "บน/ล่าง" แบบคาน
เหล็กเสริมพิเศษ	ก่อนจะตัดสินใจว่าเหล็ก "additional" อยู่บนหรือล่าง ให้ดูเส้นชี้ (leader line) จริง ๆ ไม่ใช่ดูแค่ถ้อยคำบนป้าย เพราะในชุดแบบชุดนี้ ป้ายที่หน้าตาเหมือนกันเป๊ะ กลับชี้ไปคนละด้านในมาร์คต่างกัน

บล็อก 5 OUTPUT DISCIPLINE — วินัยการตอบ ~20% ของ prompt

กฎ	ความหมาย
รวมรายการซ้ำ	<code>element_id</code> เดียวกันที่โผล่หลายครั้งในหน้าเดียว โดยตำแหน่งไม่ทับกัน → รวมเป็น entry เดียว (บวก <code>count</code> เข้าด้วยกัน ต่อ <code>grid_refs</code> เข้าด้วยกัน)
ยกเว้นตารางหลายชั้น	ตาราง schedule ที่มีหลายชั้น ให้เก็บ <code>element_id</code> เดิมแยกเป็นคนละ entry ต่อชั้น โดยใช้ฟิลด์ <code>level</code> ห้ามฝังเลขชั้นลงไปใน <code>element_id</code> เช่นห้ามเปลี่ยน <code>B1</code> เป็น <code>B1-F2</code>
1 entry ต่อ 1 ช่วงคาน	คานหนึ่งช่วงกริดถึงกริด = 1 entry ห้ามจับกลุ่มคานมาร์คเดียวกันรวมกันไว้ล่วงหน้า การจับกลุ่มเป็นงานของโค้ดปลายทาง ไม่ใช่ของโมเดล

ลำดับการอ่าน	บนล่างทีละแถว ซ้ายไปขวาทีละคอลัมน์ · ถ้าจุดเริ่มร่วมกัน ให้เอาแนวตั้งก่อนแนวนอน มีไว้ให้ผลลัพธ์เรียงเหมือนกันทุกครั้ง จะได้เทียบผลระหว่างรอบได้
ไม่ชัด = null	อะไรที่อ่านไม่ชัดให้ใส่ <code>null</code> — ห้ามเดา ห้ามแต่งค่าขึ้นมา

กฎ "ไม่ชัดใส่ null" คือกฎที่สำคัญที่สุดในบล็อกนี้

ค่าที่หายไปเราตรวจเจอและตามเก็บได้ แต่ค่าที่โมเดลแต่งขึ้นมาเองหน้าตาเหมือนของจริงทุกประการ — ไม่มีทางรู้เลยว่าอันไหนจริงอันไหนมั่ว จนกว่าจะไปเปิดแบบเทียบทีละตัว ในงานคำนวณหลัก ตัวเลขมั่วที่ดูน่าเชื่อถืออันตรายกว่าช่องว่างมาก

บล็อก 6 ปิดท้าย — 1 บรรทัด

"Reply with JSON only. No markdown fence, no commentary."

แปล: "ตอบเป็น JSON อย่างเดียว ไม่ต้องมี markdown fence ไม่ต้องมีคำอธิบาย"

โมเดลแชตถูกฝึกมาให้ "คุยกับคน" คือชอบขึ้นต้นว่า *"Here is the JSON you requested:"* แล้วครอบด้วย ````json` — พอโค้ดปลายทางเอาไปเข้า `json.loads()` ก็พังทันที บรรทัดนี้ปิดพฤติกรรมนั้น

4. ข้อยกเว้นเดียว: gridmaster (5 ตัวอย่าง)

ในชุด 403 ตัวอย่าง มี 5 ตัวที่ไม่เหมือนชาวบ้าน — หลังละ 1 ตัว เรียกว่า **gridmaster**

ตัวอย่างปกติ (398 ตัว)	gridmaster (5 ตัว)
ภาพ 1 รูป = 1 หน้าแบบ	ภาพ 2-4 รูป = ทุกหน้าของบ้านหลังนั้นที่มีเส้นกริด
ตอบสิ่งที่อยู่ในหน้านั้น	ตอบ ตารางกริดรวมของทั้งบ้าน
prompt = <code>PROMPT_SHORT</code>	prompt = <code>PROMPT_SHORT</code> + <code>GRID_SUFFIX</code> ต่อท้าย

ส่วนที่ต่อท้ายคือ:

```
These images are every page of one house that shows grid lines. Build the single house-wide grid master: all named grids plus every dummy grid, each with an id and pos_m read from a printed dimension line.
```

แปล: "ภาพเหล่านี้คือทุกหน้าของบ้านหลังเดียวกันที่แสดงเส้นกริด ให้สร้างตารางกริดรวมของทั้งบ้านออกมา 1 ชุด — ทั้งกริดที่มีชื่อและ dummy grid ทุกเส้น แต่ละเส้นต้องมี id และ `pos_m` (ตำแหน่งเป็นเมตร) ที่อ่านมาจากเส้นบอกระยะที่พิมพ์อยู่บนแบบ"

5 ตัวนี้เป็นสาเหตุที่ต้องขยาย `MAX_LENGTH` จาก 9,216 เป็น 24,576

1 ภาพกินราว 5,120 token · gridmaster บางตัวมี 4 ภาพ = ~21,000 token ถ้าไม่ขยายเพดาน ตัวเทรนจะตัดท้ายทิ้งซึ่งท้ายคือตัวคำตอบ `JSON` พอดี = สอนโมเดลด้วยคำตอบที่ถูกตัดครึ่ง และ 5 ตัวนี้เป็นสัญญาณเดียวทั้งชุดที่สอนเรื่อง dummy grid ระดับทั้งบ้าน

5. ปรัชญาเบื้องหลัง — ทำไม prompt หน้าตาแบบนี้

5.1 ความยาวแต่ละบล็อกถ่วงน้ำหนักด้วยข้อมูลจริง ไม่ใช่เขียนตามใจ

ตอนสร้าง prompt (2026-07-20) มีการไปนับจริงจาก `workmen's_diary/` ย้อนหลัง 3 สัปดาห์ ว่าแต่ละหัวข้อถูกพูดถึง / ถูกแก้กี่ครั้ง แล้วให้พื้นที่ใน prompt ตามนั้น

หัวข้อ	ถูกพูดถึงใน diary	พื้นที่ใน prompt
--------	-------------------	------------------

dummy grid + ปลายคาน	48 ครั้ง	ยาวที่สุด — 3 anti-pattern + ข้อ ยกเว้น
รูปแบบ grid_ref	39 ครั้ง	สูง
main_bar บน/ล่าง/กลาง	23 + 15 + 12	สูง
confidence / ตารางราคา / join specs	≤ 16	0 — ตัดทิ้งทั้งหมด
รวมรายการ / นับเสา / ลำดับ	≤ 6	สั้น บรรทัดเดียว

เหตุผล: เวลาที่เราลงไปนั่งรีวิวจริง คือสัญญาณที่ตรงที่สุดว่าโมเดลพลาดตรงไหนบ่อย เรื่องที่ต้องกลับไปแก้ 48 ครั้งใน 3 สัปดาห์ ควรได้พื้นที่เตือนมากกว่าเรื่องที่แทบไม่เคยผิด

5.2 ไม่ยัดสเปคเต็มลงไป

ในโค้ดมีโหมด `CFG.PROMPT_MODE = 'full_schema'` ที่จะเอาไฟล์ `primary_rawjson_schema.md` ทั้งไฟล์ (~6,000 token) มาแปะไว้หน้า prompt

แต่ไม่ได้ใช้ — `stats.json` บันทึกไว้ว่า `"PROMPT_MODE": "short"`

- 6,000 token × 403 ตัวอย่าง = ความยาวลำดับงานขึ้นมหาศาล ทั้ง VRAM และเวลาเทรน
- สเปคเต็มมีทั้งประวัติ เหตุผล ตัวอย่างที่เลิกใช้แล้ว — ส่วนใหญ่ไม่ใช่กฎที่ต้องท่อง
- ที่โมเดลต้องรู้จริง ๆ คือกฎที่มันเคยพลาด ซึ่งย่อลงได้เหลือ ~1,000 token

5.3 ตัด specs ข้ามหน้าออกจากคำตอบ ตั้งแต่ตอนสร้างชุดข้อมูล

ในข้อมูลต้นทาง บล็อก `specs{}` เกือบทั้งหมดมี `spec_source` ชี้ไปหน้าอื่น เช่น หน้า 33 (ฝั่งคาน) เอาขนาดเหล็กมาจากหน้า 37 (รูปตัดคาน)

แปลว่าตัวเลขพวกนั้น ไม่ได้พิมพ์อยู่บนภาพหน้า 33 เลย — ถ้าเทรนให้โมเดลตอบ ก็คือสอนให้มันมโนตัวเลขเหล็กตรง ๆ งานเชื่อม spec กับตำแหน่งเป็นหน้าที่ของโค้ดปลายทาง ไม่ใช่ของโมเดล

กระทบ 10 ตัวอย่าง · เปิดกลับได้ที่ `CFG.DROP_CROSS_PAGE_SPECS = false`

5.4 prompt เป็นภาษาอังกฤษ แต่คำตอบเป็นไทย

prompt เขียนอังกฤษล้วน ส่วนคำตอบมีภาษาไทยเต็มไปหมด (`raw_text` , `description` , ชื่อ sheet) ไม่ใช่ความไม่สม่ำเสมอ — คำสั่งเป็นภาษาที่โมเดลถูกฝึกมาเยอะที่สุด ส่วนเนื้อหาที่คงภาษาตามต้นฉบับบนแบบ

6. กฎตอนเรียกใช้จริง (inference)

6.1 ต้องใช้ prompt ตัวเดิมเป๊ะ

โมเดลไม่ได้เรียนแค่ "อ่านแบบ" แต่เรียนว่า "พอเจอข้อความชุดนี้ให้ตอบแบบนี้" เปลี่ยน prompt = เปลี่ยนสิ่งที่มันจำได้ ผลจะตกทันทีโดยไม่มีอะไรฟุ้ง

- ต้นฉบับ: `PROMPT_SHORT` ใน `build_dataset.js`
- สำเนาข้อความล้วนสำหรับก๊อปปี้ไปใช้:

```
tune_ai/t01/ai_output_บ้าน_ใหญ่_1ชั้น_01/_prompt_short.txt
```

6.2 ต้องปิดโหมด reasoning ทุกครั้ง

บ๊ักที่ทำให้ Phase 7 วัดผลได้ 0/3 ตอนแรก — ทั้งที่โมเดลไม่ได้พัง

`apply_chat_template(..., add_generation_prompt=True)` ถ้าไม่ใช่

`enable_thinking=False` ตัว template จะแทรกแท็ก `<think>` เปล่า ๆ ที่ยังไม่ปิด เข้าไปตรงจุดที่โมเดลจะเริ่มตอบ

โมเดลตระกูลนี้ถูกฝึกมาให้คิดออกเสียง มันก็เลยเขียน chain-of-thought ยาว ๆ ตรงนั้นจริง ๆ ("Here's a thinking process: 1. Analyze... 2. Scan...") จนกิน token หมดโควตา ยังไม่ทันถึง JSON ก็ถูกตัดจบแล้ว → วัดได้ JSON valid 0%

ตัวเทรนไม่เคยได้รับผลกระทบ — data collator เรียก template ด้วย `add_generation_prompt=False` (บทสนทนามีคำตอบครบอยู่แล้ว ไม่ต้องแทรกแท็ก) เป็นปัญหาฝั่ง inference ล้วน ๆ

วิธีปิด แล้วแต่ว่าเรียกทางไหน:

```
# ฝั่ง Python (eval_fields.py / export_gguf.py / train_qwen36.py)
tokenizer.apply_chat_template(msgs, add_generation_prompt=True,
enable_thinking=False)
```

```
# ฝั่ง llama.cpp ตอนรันบนเครื่องตัวเอง
```

```
llama-cli --model Qwen3.6-35B-A3B.Q4_K_M.gguf --mmproj mmproj-F16.gguf \
--image "หน้าแบบ.png" -p "$PROMPT_TEXT" \
--n-gpu-layers 999 --ctx-size 8192 -st --temp 0 \
--chat-template-kwarg '{"enable_thinking":false}'
```

หมายเหตุ: `--ctx-size 8192` ก็จำเป็น — โมเดลนี้รองรับ context สูงสุด 262,144 token ถ้าไม่กำหนด llama.cpp จะจอง KV-cache ตามค่าสูงสุดจนแรมเครื่องเหลือ ~0.1GB

7. สรุปสั้น ๆ

หัวข้อ	สรุป
จำนวน prompt	ตัวเดียว ใช้กับทั้ง 403 ตัวอย่าง (+ ส่วนต่อท้ายพิเศษ 5 ตัว)
ความยาว	~1,022 token
ภาษา	อังกฤษล้วน (คำตอบเป็นไทยได้ตามต้นฉบับ)
โครงสร้าง	6 บล็อก · บล็อกเรื่องกริดกินพื้นที่ ~40%
หลักการเขียน	ถ่วงน้ำหนักตามจำนวนครั้งที่เคยผิดจริง + เขียนเป็นข้อห้ามที่ชัด
สิ่งที่ตัดออก	สเปคเต็ม 6k token · specs ข้ามหน้า · บันทึกการทำงาน (ชื่อคน/วันที่)
ข้อบังคับตอนใช้	ใช้ prompt เดิมเป๊ะ + ปิด <code>enable_thinking</code> เสมอ

ผลที่ได้จริงในรอบ t01

ตัวชี้วัด	ก่อนเทรน	หลังเทรน
JSON ใช้งานได้ (valid)	0%	90%
ระบุ view ถูกต้อง	—	70%
เก็บ element ได้ครบ (recall)	0%	28.2%
element_id ตรงเป๊ะ	—	100% (48/48)

อ่านตัวเลขนี้แบบตรงไปตรงมา: **รูปทรงคำตอบใช้ได้แล้ว แต่ความครบยังไม่พอ** โมเดลออก JSON ถูกโครง อ่านชื่อมาร์คไม่ผิดเลย แต่ยังมีช่องข้ามชิ้นส่วนไปเกินครึ่ง และยังตอบเกิน 78 ชิ้นที่ไม่มีอยู่ในเฉลย

ส่วนนี้แก้ด้วย prompt ไม่ได้แล้ว — ต้องแก้ที่ข้อมูล (ปัจจุบันราว 83% ของคำตอบในชุดยังไม่มีคนตรวจ โมเดลจึงเรียนความผิดพลาดเดิมของตัวเองไปด้วย)

`tune_ai/t01/data_before_tune/build_dataset.js` (ตัว prompt + ตัวประกอบชุดข้อมูล) ·

`No_touch_box/docs/rule_of_tune.md` (กฎการแต่งข้อมูล)